

Arbitrary Precision Arithmetic Library in Java

Vansh Gupta
CS24BTECH11028

May 2025

1 Introduction

This report documents the development of an arbitrary-precision arithmetic library in Java for the Software Development Fundamentals (SDF) Project (CS1023). The library supports operations on integers (**AInteger**) and floating-point numbers (**AFloat**), handling numbers of any size with up to 30 digits of precision for floating-point division. It includes a command-line interface via **MyInfArith**.

2 Structure of my Directory

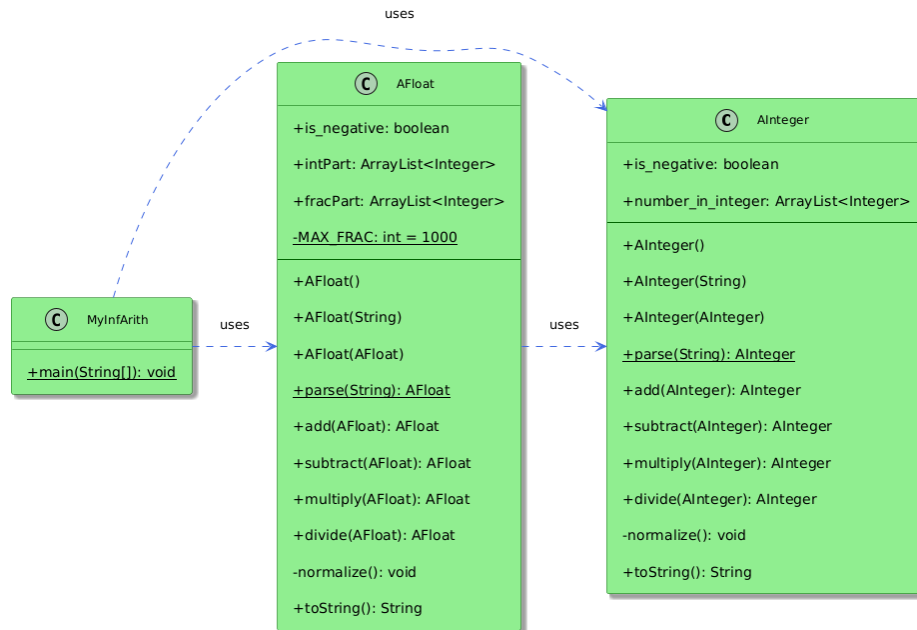
2.1 Class Structure

The library is organized in the `arbitraryarithmetic` package with two classes and one class outside with which I am managing:

- **AInteger**: Manages arbitrary-precision integers using an `ArrayList<Integer>` to store digits.
- **AFloat**: Extends the concept to floating-point numbers, with separate lists of `intPart` and `fracPart`.
- **MyInfArith**: Provides a command-line interface to perform operations based on user input.

2.2 UML Class Diagram

Below is a UML class diagram of the library to have a visual representation of the structure of a system, showing its classes, attributes, methods and the relationships between them.



UML Class Diagram for ArbitraryArithmetic Library

2.3 Main Logic

The arbitrary-precision arithmetic library uses lists to implement addition, subtraction, multiplication, and division. Input strings are converted into digit lists and processed using traditional arithmetic methods. Addition and subtraction iterate through digits, managing carry and borrow in a loop. Multiplication accumulates partial products into a zero-filled array, while division employs repeated subtraction to compute quotient digits iteratively. This list-based approach ensures accurate handling of numbers of any size.

2.4 Details about Implementation

- **Storage:** Digits are stored in `ArrayList<Integer>` to allow unbounded size.
- **Normalization:** Methods like `normalize()` remove leading/trailing zeros to optimize storage and computation.
- **Addition:** Did addition by using the digit-by-digit method with checking carry in each step and saved the result in the list itself later converted it to string.
- **Subtraction:** Did subtraction by using the digit-by-digit method with checking borrow in each step and saved the result in the list itself later converted it to string.
- **Multiplication:** Did multiplication using a list and saved the result in a list itself later converted it to a string.
- **Division:** `AFloat.divide` pads the quotient to ensure 30 digits of precision, addressing issues like `StringIndexOutOfBoundsException` and normal division by using the subtraction method.

3 How to Use the Library

3.1 As a Command-Line Tool

1. Compile the project using Ant: `ant compile`
2. Create a JAR file: `ant jar`
3. To run some test cases: `ant test`
4. Run the program with arguments:
 - a. `ant run` or just `ant`
 - This compiles the program and runs some predefined test cases.
 - b. `java -jar arbitraryarithmetic/aarithmetic.jar <type> <operation> <num1> <num2>`
 - Runs the program using the generated JAR file.

- c. `python3 coderunner.py <type> <operation> <num1> <num2>`
– Executes the program via the Python script.

Note: Ensure the JAR file exists or create it using `ant jar`.

d. **Using Docker:**

- Build the Docker image:
`docker build -t vansh132005/sdfproject .`
- Run the container with arguments:
`docker run vansh132005/sdfproject:latex <type> <operation> <num1> <num2>`

5. Example: `java -jar arbitraryarithmetic/aarithmetic.jar.jar float div 5.5 2`

3.2 As a Library

Import the JAR file into your project and use the `AInteger` and `AFloat` classes directly.

4 Limitations

- Floating-point division assumes `MAX.FRAC = 30`; higher precision requires adjusting this constant.
- Performance may degrade with extremely large numbers due to list-based storage.
- No support for advanced mathematical operations (e.g., square root, exponentiation).

5 Verification Approach

The implementation was tested against the provided public test cases, such as:

- `java MyInfArith float div 5.5 2 → 2.75`
- `java MyInfArith int div 2 0 → Division by zero error`

6 Key Learnings

- Importance of proper digit management in arbitrary-precision arithmetic.
- Handling edge cases like division by zero and precision limits.
- Using Ant for build automation simplifies project management.
- How to manage such a big project with these many tools and languages such as latex, git and github, ant, docker, python, java, UML.